JOIV : Int. J. Inform. Visualization, 10(3) - May 2026 1329-1333

INTERNATIONAL JOURNAL ON INFORMATICS VISUALIZATION

journal homepage : www.joiv.org/index.php/joiv

Achieving Sub-60ms Link Failure Recovery in Software-Defined Networks

Satria Akbar Mugitama [a,*], Hilal H. Nuha [a], Hassan Rizky Putra Sailellah [a], Goh Khang Wen [b], Khairul Amin Mohamad Sukri [c]

[a] *School of Computing, Telkom University, Bandung, Indonesia*
[b] *Faculty of Data Science and Information Technology, INTI International University, Nilai, Malaysia*
[c] *Faculty of Computer Science and Information Technology, Universiti Tun Hussein Onn Malaysia, Batu Pahat, Malaysia*
*Corresponding author: *sbg@teluapp.org*

***Abstract*—Software-Defined Networking (SDN) represents a fundamental shift in network architecture, transitioning from hardware-centric configurations to a more flexible, programmable paradigm. At its core, SDN decouples the system's intelligence—the control plane—from the underlying hardware responsible for moving traffic—the data plane. This separation was popularized by the OpenFlow protocol, originally developed at Stanford University, which provides the standardized communication interface between these two layers. However, as networks become increasingly centralized, reliability becomes a critical concern: specifically, how the system maintains connectivity when a physical link fails. To address these reliability gaps, this research developed and evaluated a prototype algorithm designed to manage link failures more efficiently than standard OpenFlow recovery mechanisms. The study focused on two primary metrics: line changeover time (the speed at which the network reroutes traffic) and system overhead. Experimental results revealed that the prototype achieved an average recovery time of 59 ms with a communication overhead of 10.1%. While this performance marks a notable improvement over existing OpenFlow methods, it highlights a persistent challenge in the field: the lack of a "carrier-grade" standard. To be considered truly resilient for mission-critical telecommunications, a network must typically recover from failure in under 50 ms. Although the prototype currently sits 9 ms above this threshold, its superior performance relative to legacy SDN methods suggests that refining the algorithm's logic or reducing its computational footprint could eventually bridge the gap, bringing programmable networks closer to industrial-grade reliability.**

***Keywords*—Software-defined networking; OpenFlow; link failure; process innovation.**

Manuscript received 18 Dec. 2025; revised 22 Mar. 2026; accepted 9 May. 2026. Date of publication 31 May. 2026.
International Journal on Informatics Visualization is licensed under a Creative Commons Attribution-Share Alike 4.0 International License.

I. INTRODUCTION

Software-Defined Networking (SDN) is a computer networking approach where the network control (control-plane) functions to handle the determination of forwarding traffic separately from the data-plane. SDN offers convenience in managing network devices by simply setting the control-plane software. The control plane software in SDN manages multiple data planes on the network, such as routers, switches, and other middlebox devices, through the Application Programming Interface (API) [1]. One of the APIs currently in use is OpenFlow, developed by Stanford University. Several controllers can run OpenFlow, namely the Python-based POX and the C-based NOX. POX is a controller suitable for research and has been extensively evaluated for its ability to enable rapid recovery from link failures using various fault-tolerance techniques [2]. Furthermore, SDN's flexibility has made it a cornerstone for smart community infrastructure and IoT services, necessitating robust management frameworks [3].

A redundant topology is needed to support high availability on the networks in conventional networks. In layer 2 networks, topology redundancy can cause broadcast storms and frame-looping problems due to broadcasts sent to each port by the switch. This problem can be solved by using the Spanning Tree Protocol (STP) 802.1, which will prevent frame-loop by blocking links/paths as backups. However, problems arise when the primary link fails. In STP, it takes more than 30 seconds to form a new link [4]. This causes the Carrier-grade standard to not be met, namely to repair (recovery) link failures of no more than 50 milliseconds on an Ethernet network [5]. To address this, recent studies have

explored proactive multi-controller approaches that can reduce recovery time to approximately 10 milliseconds [6]. Additionally, moving recovery logic to the data plane via network programmability has shown potential to achieve rerouting speeds that are three orders of magnitude faster than traditional controller-centric methods [7].

POX has a failover mechanism, namely L2 Learning Switch and L2 Multi. It uses a discovery module that sends Link Layer Discovery Protocol (LLDP) packets periodically [8]. In the OpenFlow 1.0 standard [9], some messages are sent asynchronously between the switch and controller to detect a port change. Using port-status messages in the link-failure-handling prototype algorithm is expected to reduce the time required to handle link failures. Recent advancements also suggest that scalable resilience can be achieved through programmable data planes, allowing for efficient multi-failure recovery without excessive control-plane overhead [10].

In this paper, an OpenFlow SDN network will be simulated using the Mininet emulator to determine the performance of the link failure handling mechanism. The parameters analyzed are recovery time and overhead. The final result of this paper is expected to determine the performance of the L2 Learning Switch, L2 Multi, and prototype algorithm methods in handling link failures.

II. Materials and Methods

A. Software-Defined Networking

Software-Defined Networking (SDN) is a new paradigm in networking. In a conventional network, the control plane and data packet forwarding devices are combined into a single network device, such as a switch or router. Meanwhile, the SDN network physically separates the data-packet router, which runs on a switch or router device, from a controller that can manage several devices at once.

B. Software-Defined Networking

OpenFlow is considered to be one of the first software-defined networking (SDN) standards. OpenFlow is a communication protocol that enables SDN/OpenFlow controllers to interact with the forwarding plane and adjust network forwarding. So OpenFlow is better for adapting to changing business requirements (business requirements) [11].

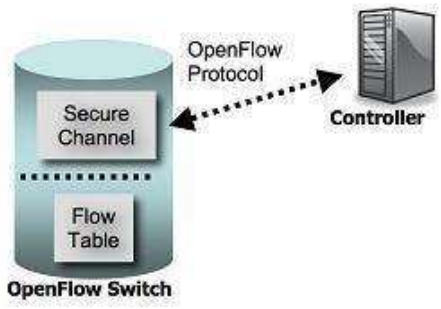


Fig. 1 OpenFlow 1.0 [3]

The SDN controller is the brain of the network. The controller is a strategic control center that transmits information to the switch or router device at the bottom (via southbound APIs) and the application layer and business logic at the top (via northbound APIs). SDN controller development uses common applications, such as OpenFlow and Open Virtual Switch Database (OVSDB) [6]. Now some organizations are developing SDN networks, while SDN controllers are handled by the federation that manages SDN domain controllers.

The OpenFlow standard is based on modern switches or routers having Forwarding Information Bus (FIB) implemented using Ternary Content Addressable Memory (TCAM). FIB can be programmed using the OpenFlow protocol by adding/deleting (adding/deleting) entries to the flow table.

FlowTable is an abstraction of FIB. In an OpenFlow network, all logic is centralized in a single system called the OpenFlow controller. The controller manages OpenFlow switches. So the OpenFlow switch consists of a FlowTable which functions to search and forward packets, and a Secure Channel to communicate with an external controller [1].

C. POX

POX is an open-source platform used for software-defined networking (SDN) controller applications based on Python. Using POX allows rapid development and prototyping, so POX is becoming more widely used than NOX [12].

POX has several features, namely:

- Pythonic OpenFlow interface.
- Provide a sample code that can be reused (Reuseable).
- Runs anywhere can be installed with the PyPy runtime.
- Specifically targeted for Linux, Mac OS, and Windows.
- Topology discovery.
- Supports GUI and visualization tools like NOX.

Better performance than applications on NOX, the program was written in Python. POX has the following components.

D. L2-Learning

L2-learning is one of the components contained in POX. This component is tasked with making the Openflow switch work as an L2 learning switch by mapping the MAC address and port of the Openflow switch to the controller. The mapping address is obtained by monitoring incoming packets on the switch and recording the packet's MAC address on the controller [13].

Packet delivery depends on matching the packet's destination address to the address in the MAC table on the controller (which contains the destination address and switch port). When there is a match between the packet's destination address and the MAC list, the controller adds a FlowEntry to the OpenFlow switch so the switch can forward the packet through the correct port. However, when no match is found, the packet's destination address will be stored in the MAC list, and the packet will be broadcast to all ports. Packets with broadcast or multicast destinations are broadcast directly without being stored in the MAC table.

The OpenFlow switch supports the port-flood blocking feature by prohibiting sending broadcast packets. The POX controller has a Spanning Tree module that can enable this feature, eliminating loops in redundant topologies that cause broadcast storms and frame looping.

L2 learning does not implement aging timer logic. Aging timer logic [14] is a mechanism in which, when the switch learns an origin address, it records the time (timestamp) in the entry. Each time the switch receives a frame from the source,

it updates the timestamp. When the switch does not receive a frame from the source during the aging timer expiration, the switch deletes the entry from the MAC table.

Due to the absence of aging timer logic, L2 learning cannot delete entries from its MAC table upon link failure. But there is a way to update entries when packets are received from some other ports. This allows L2 learning to broadcast. There are two cases of packets being broadcast. First, when the destination address is not in the MAC table. Second when the destination address is broadcast or multicast. In the absence of aging timer logic, the first case is impossible because the entry already exists in the MAC table [15], but the second case can occur.

When a data packet is sent across the network, Address Resolution Protocol (ARP) also runs in parallel if there is no static, permanent ARP entry on the node. The ARP_request packet uses a broadcast address, allowing L2 learning to form a new path. An ARP request with a broadcast address is sent when the "node reachable time" on the ARP entry expires [16]. This means that the formation of a new path in L2 learning depends on the delivery time of the ARP request with the broadcast address. In a link failure, incorrect flow entries will continue to be added to the OpenFlow switch if the host does not send an ARP request [17].

Hosts with permanent entries in the ARP table will not send ARP requests. The absence of packets sent with broadcast or multicast addresses prevents L2 learning from handling link failures.

E. L2 Multi

L2 Multi is a method of determining the shortest path between end hosts. There are four mechanisms for establishing the shortest path between hosts. The mechanisms are discovery, topology, authentication, and routing. The mechanism carried out by L2 Multi is: The first is the Discovery mechanism, which uses packet-in and packet-out messages to run discovery protocols on OpenFlow switches. This mechanism sends a packet out to send Link Layer Discovery Protocol (LLDP) on all OpenFlow switches [18]. The LLDP packet is immediately sent to the output port when the switch receives a packet out. When the switch receives an LLDP packet, it also sends a "packet in" to POX to notify of the detected path.

The second is a topological mechanism useful for storing paths (links between switches) formed as a result of the discovery. Next is the authentication mechanism that forms host or user data connected to one of the switches (host with switch). Finally, the routing mechanism determines the shortest path between registered hosts using the Floyd-Warshall algorithm (the default search algorithm in POX). The routing mechanism will calculate the shortest distance and add a flow table to each OpenFlow switch if the destination is known. The controller will forward a packet because there is no matching flow in the flow table. If the authentication mechanism does not know the source packet's address, the host address will be registered in the switch's host table. However, if it is unknown in the host--switch table, the packet will be broadcast [19].

The link failure detection mechanism in L2 Multi is very dependent on the "LinkEvent" discovery module and the timeout interval for making topology changes or convergence. The discovery module sends LLDP packets periodically. If the controller does not receive LLDP packets within a specified time, it detects a broken link. Deleting all flow entries on the switch will cause the switch to send packets to the controller because there is no match between the flow entries and the incoming packets. This event will trigger a Packet In event on the controller, and the controller will form a new path according to the data transmission mechanism in L2 Multi, using valid switch path data.

F. Algorithm modified

POX has several methods of packet delivery at layer 2, namely hub, L2 learning, and L2 multi. L2 multi is the fastest method of handling link failures. However, L2 Multi relies on a discovery module that uses periodically sent LLDP packets. LLDP packet delivery affects the link failure detection speed on L2 Multi. The faster the delivery time interval for LLDP packets, the better the handling of link failures, but it increases the overhead on the network. In Openflow 1.0, a port state change detection mechanism can detect link breaks without waiting for periodic LLDP packets to be sent. In this study, modifications were made to L2 multi by adding port status detection.

III. Results and Discussion

A. Testing Scenario

The simulation carried out in this study uses a network topology as shown in Figure 2, which consists of two host computers, six switches, and one controller.

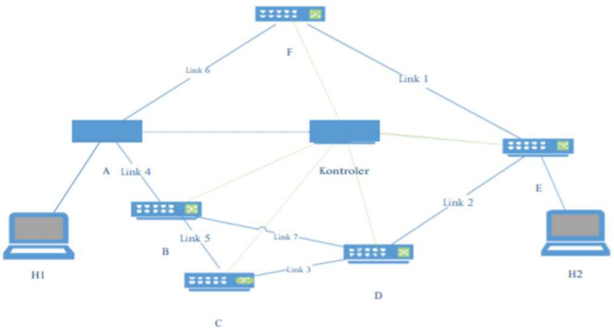


Fig. 2 Network topology

B. Scenario 1

Scenario one carried out overhead tests on the controller and the network. Two measurements were taken: overhead on the infrastructure side and on the controller. On the infrastructure side, there is overhead in the data path network. On the controller side, the overhead occurs between the switch and the controller. In this scenario, each method is applied to the controller by activating the Discovery and Spanning Tree modules. Under normal conditions, the link timeout is set to a value of 10 seconds. In this paper's test, the discovery module will be set with a link timeout of 1 to 10 seconds. Host 1 will ping at 10-millisecond intervals. The calculation of infrastructure overhead is recorded using Wireshark on host 1 and the switch.

Packets to be recorded are ping packets, as the primary information sent, and LLDP packets, as additional

information in the network infrastructure. The formula obtains the value of the overhead of the infrastructure:

$$\gamma = \frac{total\ packet\ size - total\ data\ size}{total\ packet\ size} \times 100\% \quad (1)$$

The calculation of controller overhead is recorded on host 1 and controller. The logged packets are the main information ping packet and all OpenFlow packets that pass through the controller. The formula obtains the value of the controller overhead c:

$$\gamma_c = \frac{total\ controller\ packet\ size - total\ data\ size}{total\ controller\ packet\ size} \times 100\% \quad (2)$$

Overhead will increase if at the time of the link timeout is reduced. This is due to the discovery module having less time to detect link failures. It will shorten the delivery interval for LLDP packets on the network, prompting the OpenFlow protocol to send data from the switch to the controller. This will burden the controller's work if the LLDP packet delivery interval is too fast.

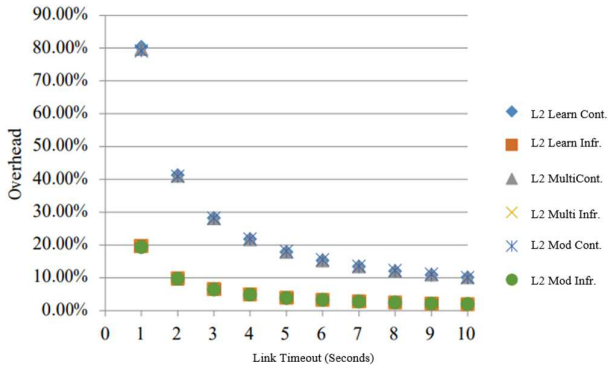


Fig. 3 Comparison graph between methods

From Figure 3, there is no significant difference in overhead between the tested methods. With a link timeout of 1 second in the L2 learning method, controller overhead is 80.37%, and infrastructure overhead is 19.74%. In L2 Multi-controller, overhead is 79.68%, and infrastructure overhead is 19.51%. In L2, the overhead controller modification is 79.17%, and the infrastructure overhead is 19.39%.

C. *Scenario 2*

There are two scenarios in which link-failure time testing is conducted for each method. Link failure repair time is required by all network switches to return to a steady state following a topology change [10]. The occurrence of link failures influences topology changes. The discovery module plays a vital role in this test because the link change detection functionality is implemented in it. The time it takes the switch to return to the ready state depends on how quickly link failure is detected. The detection speed can be set in the discovery module using the link-timeout command.

The logic used in Figure 2, with link 5, will be disconnected so that there are only two paths: line one ABDE and line two AFE. Line two will be disconnected, forcing the controller to form a path on switch ABDE (line one). Within 30 seconds, line two will be connected, and in the next 5 seconds, line one will be disconnected. This condition will force the controller to change its path.

The test is carried out by sending ICMP Ping packets from host 1 to host 2 at 10-millisecond intervals. The link-timeout value will be changed from 1 second to 10 seconds in the discovery module. Measurements are taken at the time of a link failure, namely when the ICMP_req from host 2 (destination) is not received by host 1 (source). When the controller has changed the path, host 1 will receive back ICMP_req from host 2. The difference between the time when ICMP_req packets were received (T_1) and the time when ICMP_req started not being received (T_0) is the time at which link failure repair T= T_1-T_0 .

Link timeout only affects L2 Multi. At the same time, L2 Learning and L2 Modification have no effect. This is because L2 learning relies on link timeouts in the discovery module to detect link failures. Whereas in L2 learning, the handling of link failures depends on the ARP request made by the host, and in L2 modification, the port status of the OpenFlow switch determines how link failures are handled.

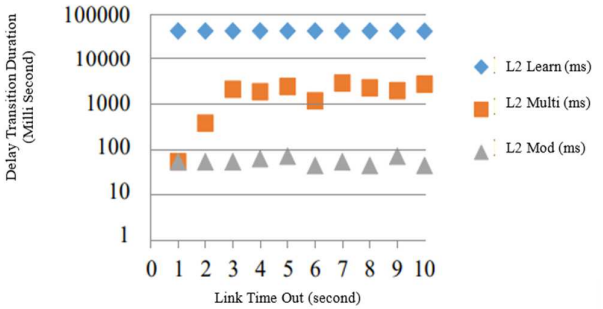


Fig. 4 Link failure handling time graph

Figure 4 shows that L2 multi affects the discovery module's link timeout. At the same time, L2 learning and L2 modification have no effect. L2 learning has an average link failure handling time of 48426.4 milliseconds (48.4 seconds). While the L2 Mod has an average handling time of 62 milliseconds. Meanwhile, for L2 Multi, the fastest time with a 1-second link timeout is 60 milliseconds, and the largest value with a 10-second link timeout is 3.1 seconds.

D. *Scenario 3*

Scenario 3 tests by increasing the number of switches up to three times. The analysis was carried out by observing the effect of the number of switches with the link failure repair time. Switches will be added to each path to form a ring topology. A switch will be added between link AF and link AB. The test is carried out by sending ICMP Ping packets from host 1 to host 2 at 10-millisecond intervals. Then the process of changing the path is carried out as in scenario 2.

Table 1 shows that the increase in the number of switches between six and sixteen did not significantly affect the link failure handling time. In L2 multi, the average result is 3986 milliseconds, while the modified L2 results in an average time of 59 milliseconds.

TABLE I
LINK FAILURE RECOVERY TIME (MS) FOR DIFFERENT NUMBERS OF SWITCHES

Method (in milliseconds)	**Number of Switches**										
	6	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**
L2 multi	3905	3585	3405	4205	4797.5	3720	4520	3670	4225	3835	3986.8
L2 Modes	62	62.5	52.5	65	50	55	52.5	65	52.5	62.5	67.5

IV. CONCLUSIONS

Based on the objectives and results of the tests and analyses, it conducted link-failure testing on SDN OpenFlow using the L2 Learning, L2 Multi, and modified L2 prototype algorithms. It can be concluded that the modified L2 performs best in handling link failures, achieving the fastest time of 59 milliseconds and an overhead of only 10.1%, as detailed below. The modified L2 method has a shorter average time to handle link failures than L2. Learning and L2 Multi, with an average speed of 59 milliseconds and a maximum of 16 switches. The modified L2 method has a lower overhead of 70.3% than the L2 multi method with the same handling speed. Using the modified L2 method yielded an average time of 59 milliseconds. It is hoped that the presented method can be expanded to a number of application fields, including [20]-[30].

REFERENCES

[1] S. Sharma, D. Staessens, D. Colle, M. Pickavet, and P. Demeester, "Enabling fast failure recovery in OpenFlow networks," in *2011 8th Int. Workshop Des. Reliable Commun. Netw. (DRCN)*, Oct. 2011, pp. 164-171, doi: 10.1109/drcn.2011.6076899.

[2] A. Menaceur, H. Drid, and M. Rahouti, "Fault tolerance and failure recovery techniques in software-defined networking: A comprehensive approach," *J. Netw. Syst. Manage.*, vol. 31, no. 4, Sep. 2023, doi: 10.1007/s10922-023-09772-x.

[3] R. Chaudhary, G. S. Aujla, N. Kumar, and P. K. Chouhan, "A comprehensive survey on software-defined networking for smart communities," *Int. J. Commun. Syst.*, vol. 38, no. 1, Jul. 2022, doi:10.1002/dac.5296.

[4] M. Pustylnik, M. Zafirovic-Vukotic, and R. Moore, "Performance of the Rapid Spanning Tree Protocol in ring network topology," 2007. [Online]. Available: http://w3.siemens.com/mcms/industrial-communication/en/rugged-communication/Documents/rstp-in-ring-network-topology-en.pdf.

[5] MEF, "Metro Ethernet Forum," Jun. 2013. [Online]. Available: https://www.metroethernetforum.org/carrier-ethernet-services/carrier-ethernet-and-ce-2-0.

[6] W. O. Delfino et al., "Proactive fault-recovery on distributed software-defined network controllers," *J. Netw. Syst. Manage.*, vol. 33, no. 3, Jun. 2025, doi: 10.1007/s10922-025-09944-x.

[7] J. A. Marques, K. Levchenko, and L. P. Gaspary, "Responding to network failures at data-plane speeds with network programmability," in *NOMS 2023-2023 IEEE/IFIP Netw. Oper. Manage. Symp.*, May 2023, pp. 1-10, doi: 10.1109/noms56928.2023.10154329.

[8] W. Rafique et al., "Complementing IoT services through software defined networking and edge computing: A comprehensive survey," *IEEE Commun. Surv. Tutor.*, vol. 22, no. 3, pp. 1761-1804, 2020, doi: 10.1109/comst.2020.2997475.

[9] Y. Zhang, L. Cui, W. Wang, and Y. Zhang, "A survey on software defined networking with multiple controllers," *J. Netw. Comput. Appl.*, vol. 103, pp. 101-118, Feb. 2018, doi:10.1016/j.jnca.2017.11.015.

[10] M. Alsaeedi, M. M. Mohamad, and A. A. Al-Roubaiey, "Toward adaptive and scalable OpenFlow-SDN flow control: A survey," *IEEE Access*, vol. 7, pp. 107346-107379, 2019, doi:10.1109/access.2019.2932422.

[11] Z. Li, Y. Hu, J. Wu, and J. Lu, "P4Resilience: Scalable resilience for multi-failure recovery in SDN with programmable data plane," *Comput. Netw.*, vol. 208, p. 108896, May 2022, doi:10.1016/j.comnet.2022.108896.

[12] Open Networking Foundation, "OpenFlow Switch Specification, Version 1.0.0." [Online]. Available: http://archive.openflow.org/documents/openflow-spec-v1.0.0.pdf.

[13] M. McCauley, "About POX," *POX Repository*. [Online]. Available: http://www.noxrepo.org/pox/about-pox/.

[14] D. Cabarkapa and D. Rancic, "Performance analysis of Ryu-POX controller in different tree-based SDN topologies," *Adv. Electr. Comput. Eng.*, vol. 21, no. 3, pp. 31-38, 2021, doi:10.4316/aece.2021.03004.

[15] D. Muelas, J. Ramos, and J. E. L. de Vergara, "Assessing the limits of Mininet-based environments for network experimentation," *IEEE Netw.*, vol. 32, no. 6, pp. 168-176, Nov. 2018, doi: 10.1109/mnet.2018.1700277.

[16] Minnesota Supercomputer Center, "IP over ATM: overhead analysis," Aug. 12, 1994. [Online]. Available: http://www.sonic.net/support/docs/ip-atm.overhead.pdf.

[17] S. Shimokawa, T. Kanaoka, Y. Taenaka, K. Tsukamoto, and M. Lee, "SDN-based time-domain error correction for in-network video QoE estimation in wireless networks," in *Adv. Intell. Syst. Comput.*, Springer, 2019, pp. 331-341, doi: 10.1007/978-3-030-29035-1_32.

[18] A. Basu and J. Riecke, "Stability issues in OSPF routing," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 31, no. 4, pp. 225-236, Aug. 2001, doi: 10.1145/964723.383077.

[19] N. Feamster, J. Rexford, and E. Zegura, "The road to SDN," *Queue*, vol. 11, no. 12, pp. 20-40, Dec. 2013, doi: 10.1145/2559899.2560327.

[20] N. N. M. Azam, M. A. Ismail, M. S. Mohamad, A. O. Ibrahim, and S. Jeba, "Classification of COVID-19 symptoms using multilayer perceptron," *Iraqi J. Comput. Sci. Math.*, vol. 4, no. 4, pp. 100-110, Oct. 2023, doi: 10.52866/ijcsm.2023.04.04.009.

[21] K. Moorthy, A. N. Jaber, M. A. Ismail, F. Ernawan, M. S. Mohamad, and S. Deris, "Missing-values imputation algorithms for microarray gene expression data," in *Methods Mol. Biol.*, vol. 1986, Springer, 2019, pp. 255-266, doi: 10.1007/978-1-4939-9442-7_12.

[22] H. Kumar, P. J. Soh, and M. A. Ismail, "Big data streaming platforms: A review," *Iraqi J. Comput. Sci. Math.*, vol. 2, no. 1, pp. 95-100, Apr. 2022, doi: 10.52866/ijcsm.2022.02.01.010.

[23] E. H. Tusher and M. A. Ismail, "FLSTM-ESD: A hybrid approach to fuzziness based long short-term memory network for email spam detection," *Adv. Eng. Inform.*, vol. 73, p. 104597, Jul. 2026, doi:10.1016/j.aei.2026.104597.

[24] S. Armoogum et al., "A comprehensive review of cyber hygiene practices in the workplace for enhanced digital security," *JOIV: Int. J. Informatics Visualization*, vol. 9, no. 1, p. 137, Jan. 2025, doi:10.62527/joiv.9.1.3787.

[25] M. F. Afansyah and H. S. Ahmad Nawi, "The architecture of digital deception: Mapping fake news typologies, bot behaviors, and platform vulnerabilities," *Int. J. Adv. Sci. Comput. Eng.*, vol. 7, no. 3, pp. 80-84, Dec. 2025, doi: 10.62527/ijasce.7.3.257.

[26] I. Hidayat, A. W. Dani, M. Amin, I. Hermala, and Abdurohman, "Fuzzy logic and IoT-based monitoring for solar-powered precision mist irrigation," *Int. J. Adv. Sci. Comput. Eng.*, vol. 7, no. 2, pp. 53-59, Aug. 2025, doi: 10.62527/ijasce.7.2.261.

[27] M. El-Dalahmeh and A. El-Dalahmeh, "Secure key management schemes for high-mobility VANET environments," *Int. J. Adv. Sci. Comput. Eng.*, vol. 7, no. 1, pp. 22-28, Apr. 2025, doi:10.62527/ijasce.7.1.243.

[28] R. Kumar, "Genetic algorithm based secure hybrid routing technique for IoT framework," *Int. J. Adv. Sci. Comput. Eng.*, vol. 6, no. 1, pp. 13-19, Apr. 2024, doi: 10.62527/ijasce.6.1.194.

[29] N. K. Baqer, A. W. Abbas, and B. A. Salih, "Data and management traffic of IEEE 802.15.4 ZigBee-based WSN," *Int. J. Adv. Sci. Comput. Eng.*, vol. 6, no. 2, pp. 80-83, Jul. 2024, doi:10.62527/ijasce.6.2.210.

[30] A. Qusay Subhe and M. A. Shareef, "Relational reasoning in remote sensing: A review of current paradigms and the case for a co-evolutionary framework," *Int. J. Data Sci.*, vol. 7, no. 1, pp. 1-18, May 2026, doi: 10.18517/ijods.7.1.1-18.2026.